HANDS-ON LAB 03 GUIDE & COPY-PASTE PROMPTS • OMNICHANNEL RETAIL EDITION (RETAIL PULSE)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 03 (EN)

Lab 03: Retail MCP Server, Deterministic Margin Hooks & Enterprise Security Governance

Duration: 45 Minutes | **Persona**: Technical & Commercial Track | **Prerequisites**: Lab 02 Completed

1. Retail Scenario & Business Context

Even after encoding rules and skills (Lab 02), two major operational challenges remain in an enterprise retail organization:

1. **Live System Grounding via Model Context Protocol (MCP)**:

Neither engineers nor commercial category managers want to copy-paste raw SQL queries or CSV exports into chat windows. Instead, Antigravity should query the live **`data/retail_pulse.db`** SQLite database directly through structured **Model Context Protocol (MCP)** tools to inspect True ATP across nodes, audit breaching vendor OTIF contracts, simulate promotional margins before launch, and list VIP orders at risk of stockout.

2. **Deterministic Guardrails & Enterprise Governance**:

Instructions in **`.agents/rules/`** guide the LLM's reasoning, but mission-critical compliance—such as blocking someone from disabling **`enforce_floor=False`** in **`src/services/promo_pricing.py`** or hardcoding **`minimum_margin_pct`** below **`0.15`** (**`15%`**)—requires **deterministic enforcement** that executes outside the LLM's probabilistic output, alongside **Google Cloud / Gemini Enterprise governance** and **Antigravity Client Sandbox & Permission controls**.

2. Learning Objectives

In this lab, you will:

1. Complete the **Retail Pulse FastMCP Server** (**`mcp_server/retail_mcp.py`**) by implementing two high-impact MCP tools:

- **`simulate_promo_margin_guard`**: Simulates stacking a promotional campaign against a SKU or category and reports whether any SKU breaches its minimum margin floor.
- **`list_at_risk_vip_orders`**: Queries **`omnichannel_orders`** for **`AT_RISK_STOCKOUT`** orders filtered by loyalty tier (**`DIAMOND`**, **`GOLD`**) and ranks them by Customer Lifetime Value (**`customer_ltv_usd`**).

2. Register the local MCP server in **`.agents/mcp_config.json`** so Antigravity discovers and invokes all 4 retail MCP tools automatically.

3. Configure a deterministic **`PreToolUse`** lifecycle hook in **`.agents/hooks.json`** backed by **`scripts/margin_guard_hook.py`** that intercepts any **`write_file`** / **`edit_file`** tool call violating the 15% margin floor and writes an immutable local audit trail to **`data/logs/hook_compliance_audit.jsonl`**.
4. Walk through **Google Cloud & Gemini Enterprise Governance** and **Antigravity Client & Workspace Security Controls** for enterprise retail deployments.

3. Step-by-Step Instructions

Step 1: Initialize the Lab 03 Starter State

Run the Lab 03 installer to ensure Labs 01 and 02 solutions are active and load the Lab 03 starter files:

```
./labs/lab-03-mcp-and-hooks/install.sh
```

Verify that the Lab 03 test suite currently fails on the unimplemented MCP tools and missing **`.agents/mcp_config.json`** / **`.agents/hooks.json`** files:

```
./labs/lab-03-mcp-and-hooks/verify.sh
```

Step 2: Inspect the Starter MCP Server (`mcp_server/retail_mcp.py`)

Open **`mcp_server/retail_mcp.py`**. Notice that two MCP tools are already implemented:

- **`query_true_atp_inventory(sku: str = "", category: str = "") -> str`**
- **`audit_vendor_otif_contracts(only_breaching: bool = True) -> str`**

However, two tools are currently stubs returning **`{"status": "NOT_IMPLEMENTED"}`**:

- **`simulate_promo_margin_guard(promo_code: str, category: str = "", loyalty_tier: str = "SILVER") -> str`**
- **`list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND", limit: int = 20) -> str`**

Step 3: Implement the Remaining MCP Tools & Register `.agents/mcp_config.json`

Open **`labs/lab-03-mcp-and-hooks/PROMPTS.md`** and use the **Implementation Prompts** (or **Inline Edit `Cmd+I` / `Ctrl+I`**) to:

1. Implement **`simulate_promo_margin_guard`** in **`mcp_server/retail_mcp.py`** so it evaluates all SKUs in the requested category against the requested **`promo_code`** and **`loyalty_tier`** using **`evaluate_sku_promo()`**, returning **`total_skus_evaluated`**, **`floor_violations_count`**, **`requires_vp_merchandising_override`**, and per-SKU details (**`sku`**, **`product_name`**, **`unit_cost_usd`**, **`msrp_usd`**, **`effective_price_usd`**, **`margin_pct`**, **`minimum_margin_pct`**, **`floor_clamped`**).
2. Implement **`list_at_risk_vip_orders`** in **`mcp_server/retail_mcp.py`** so it queries **`omnichannel_orders`** where **`status = 'AT_RISK_STOCKOUT'`** and **`customer_loyalty_tier = loyalty_tier`** (or all tiers if **`loyalty_tier == "ALL"`**), ordered by **`customer_ltv_usd DESC`**, returning **`count`**, **`total_ltv_exposed_usd`**, and **`orders`**.
3. Create **`.agents/mcp_config.json`** registering the **`retail-pulse-mcp`** server via **`.venv/bin/python -m mcp_server.retail_mcp`**.

Step 4: Configure the Deterministic `PreToolUse` Margin Guard Hook (`.agents/hooks.json`)

Inspect **`scripts/margin_guard_hook.py`**. This script reads the tool call payload from **`stdin`** before any file modification occurs:

- If a tool call attempts to set **`enforce_floor=False`** or **`enforce_floor = False`** inside **`src/services/promo_pricing.py`**, or sets **`"minimum_margin_pct"`** below **`0.15`**, the hook exits with code **`2`** (**`BLOCKED`**) and outputs a clear compliance error message to **`stderr`**.
- Every hook evaluation (**`ALLOW`** or **`BLOCK`**) is appended as a structured JSON event to **`data/logs/hook_compliance_audit.jsonl`**.

Create **`.agents/hooks.json`** to wire **`scripts/margin_guard_hook.py`** to the **`PreToolUse`** lifecycle event for **`write_file`**, **`edit_file`**, and **`replace_file_content`**.

Test the hook directly from the terminal to verify both the block behavior and the audit log entry:

```
echo '{"tool_name":"edit_file","tool_input":{"Path":"src/services/promo_pricing.py","Rep \
  lacement":"enforce_floor=False"}}' | .venv/bin/python scripts/margin_guard_hook.py
cat data/logs/hook_compliance_audit.jsonl
```

4. Enterprise Security, Privacy & Governance Deep-Dive

When deploying Antigravity across a retail enterprise (merchandising, supply chain, store operations, and platform engineering), security architects govern the platform at **two complementary layers**:

```
+-------------------------------------------------------------------------------+
| LAYER 1: Google Cloud & Gemini Enterprise Governance (Cloud Admin Plane)      |
|  - IAM & Identity: roles/cloudaicompanion.user via Cloud Identity / Corporate SSO |
|  - Cloud Audit Logs: cloudaicompanion.googleapis.com in Google Cloud Logging  |
|  - Data Privacy: Zero Data Retention & No Model Training on customer prompts/code |
|  - Network Perimeter: VPC Service Controls (VPC-SC) & Regional Endpoints      |
|  - IP Protection: Google Cloud Indemnification & Citation Filters             |
+-------------------------------------------------------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------------------+
| LAYER 2: Antigravity Client & Workspace Security Controls (Local Execution Plane) |
|  - Tool Execution Policy: strict | request-review | proceed-in-sandbox        |
|  - Terminal Sandbox & Network Isolation: OS-level process & filesystem sandboxing |
|  - Non-Workspace File Access: Restrict read/write strictly to repo boundary   |
|  - Command Allowlist / Denylist & Browser Allowlist                           |
|  - Deterministic Lifecycle Hooks: .agents/hooks.json -> compliance audit JSONL |
+-------------------------------------------------------------------------------+
```

Layer 1: Google Cloud & Gemini Enterprise Governance

Control Area	Google Cloud / Gemini Enterprise Capability	Retail Enterprise Use Case
IAM & Seat Management	Assign **`roles/cloudaicompanion.user`** via Google Cloud IAM integrated with **Cloud Identity / Okta / Azure AD (Entra ID)** SSO groups.	Grant access separately to **`retail-platform-eng@`** and **`retail-commercial-ops@`** with centralized onboarding/offboarding.

Control Area	Google Cloud / Gemini Enterprise Capability	Retail Enterprise Use Case
Cloud Audit Logging	Enable **Admin Activity** and **Data Access** audit logs for **`cloudaicompanion.googleapis.com`** in **Google Cloud Logging**, and route via Log Sinks to BigQuery or Chronicle SIEM.	Maintain a tamper-evident enterprise audit trail of who invoked Gemini Enterprise, from which IP/device, and when.
Data Privacy & Zero Training	Customer prompts, workspace context, SQL schemas, and generated code are **never used to train shared foundation models** and adhere to strict zero-retention guarantees outside your tenant boundary.	Safely analyze proprietary vendor contracts (**`data/exports/vendor_contracts.csv`**), wholesale unit costs, and VIP customer LTV data.
VPC Service Controls (VPC-SC)	Place **`cloudaicompanion.googleapis.com`** inside a **VPC Service Controls perimeter** alongside BigQuery, Cloud SQL, and Secret Manager, with regional endpoint pinning.	Prevent data exfiltration from corporate workstations or unauthorized external networks.

Layer 2: Antigravity Client & Workspace Security Controls

Client Setting / Hook	Configuration Options	Recommended Retail Production Setting
Tool Execution Policy	**`strict`**, **`request-review`**, **`proceed-in-sandbox`**, **`always-proceed`**	Use **`proceed-in-sandbox`** or **`request-review`** so mutating shell commands require either OS sandboxing or human approval.
Terminal Sandbox & Network Policy	OS-level sandbox (**`sandbox-exec`** on macOS / namespaces on Linux) + Internet Access Toggle	Enable Terminal Sandbox and restrict external network calls so agents can only reach approved internal endpoints.
Non-Workspace File Access	Block or require confirmation for file reads/writes outside the workspace root	**Block** non-workspace access so agents cannot read **`~/.ssh`**, **`~/.aws`**, or unrelated directories on the host machine.
Command Allowlist / Denylist	Prefix allowlist (e.g., **`pytest`**, **`.venv/bin/python`**) and denylist (**`rm -rf`**, **`curl`**, **`nc`**)	Allowlist **`.venv/bin/python -m pytest`** for autonomous TDD while blocking destructive shell or network exfiltration commands.
Browser Allowlist	Domain allowlist for the **`/browser`** subagent	Restrict **`/browser`** strictly to **`http://127.0.0.1:8000`** and approved internal staging dashboards.
Deterministic Lifecycle Hooks	**`.agents/hooks.json`** (**`PreToolUse`**, **`PostToolUse`**)	Enforce commercial and security invariants (such as **`scripts/margin_guard_hook.py`**) and log every check to **`data/logs/hook_compliance_audit.jsonl`**.

5. Verification Checklist

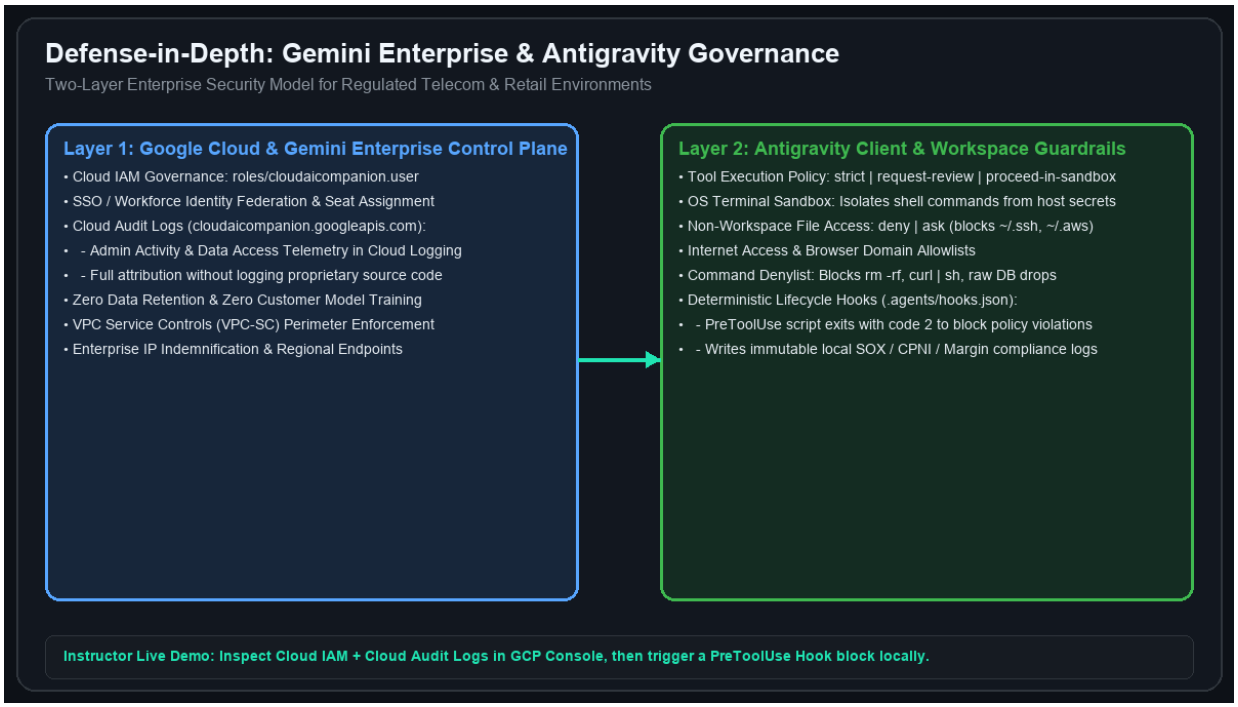
Run the Lab 03 verification script to confirm your MCP tools, **`.agents/mcp_config.json`**, and **`.agents/hooks.json`** pass all automated tests:

```
./labs/lab-03-mcp-and-hooks/verify.sh
```

Expected output:

- **`tests/test_lab03_mcp_hooks.py`** passes **5/5 tests**:
- **`test_mcp_query_true_atp_inventory`**
- **`test_mcp_audit_vendor_otif_contracts`**
- **`test_mcp_simulate_promo_margin_guard`**
- **`test_mcp_list_at_risk_vip_orders`**
- **`test_mcp_config_and_margin_guard_hook`**

Need to jump straight to the verified state? Run **`./labs/lab-03-mcp-and-hooks/apply_solution.sh`**.



LAB 03 COPY-PASTE ANTIGRAVITY PROMPTS (EN)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Lab 03 Prompt Playbook: Retail MCP Server & Deterministic Margin Hooks

Copy and paste these prompts into **Google Antigravity** as you progress through Lab 03. Each section includes prompts tailored for both the **Technical Track** and the **Commercial / Merchandising Track**.

Phase 1: Discovery & Architecture Inspection

Prompt 1A — Technical Track: Inspect the Starter MCP Server & Hook Script

```
Read @mcp_server/retail_mcp.py, @scripts/margin_guard_hook.py, and @tests/test_lab03_mcp \
  _hooks.py.
Summarize:
1. How the two existing MCP tools (`query_true_atp_inventory` and `audit_vendor_otif_con \
  tracts`) connect to `data/retail_pulse.db` and our service layer.
2. What exact JSON fields `simulate_promo_margin_guard` and `list_at_risk_vip_orders` mu \
  st return to pass `tests/test_lab03_mcp_hooks.py`.
3. How `scripts/margin_guard_hook.py` inspects `PreToolUse` payloads on stdin and writes \
   compliance events to `data/logs/hook_compliance_audit.jsonl`.
```

Prompt 1B — Commercial / Operations Track: Understand Live MCP Grounding vs. Deterministic Hooks

```
Read @mcp_server/retail_mcp.py and @scripts/margin_guard_hook.py. Explain in plain retai \
  l terms:
1. How the 4 MCP tools let a Merchandising or Supply Chain manager ask live questions ab \
  out True ATP stock, breaching vendor OTIF contracts, promo margin impact, and VIP orde \
  rs at risk without writing SQL.
2. Why we pair our `.agents/rules/retail-standards.md` guidelines with a deterministic ` \
  PreToolUse` hook (`scripts/margin_guard_hook.py`) that blocks anyone from turning off  \
  the 15% margin floor.
```

Phase 2: Planning (`/plan`)

Prompt 2 — Create an Implementation Plan for MCP Tools & Lifecycle Hooks

```
/plan Design the implementation for Lab 03 so that all 5 tests in @tests/test_lab03_mcp_ \
  hooks.py pass:
1. Implement `simulate_promo_margin_guard(promo_code: str, category: str = "", loyalty_t \
  ier: str = "SILVER") -> str` and `list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND \
  ", limit: int = 20) -> str` in @mcp_server/retail_mcp.py.
2. Create `.agents/mcp_config.json` registering `retail-pulse-mcp` using `.venv/bin/pyth \
  on -m mcp_server.retail_mcp`.
3. Create `.agents/hooks.json` registering `.venv/bin/python scripts/margin_guard_hook.p \
  y` as a `PreToolUse` hook matching `write_file|edit_file|replace_file_content`.
```

Phase 3: Inline Edit (`Cmd+I` / `Ctrl+I`)

Open **`mcp_server/retail_mcp.py`**, highlight the **`simulate_promo_margin_guard`** and **`list_at_risk_vip_orders`** stub functions, press **`Cmd+I`** (macOS) or **`Ctrl+I`** (Windows/Linux), and enter:

```
Implement `simulate_promo_margin_guard` using `evaluate_sku_promo` from `src.services.pr \
  omo_pricing` (with `enforce_floor=True`) so it returns JSON with `promo_code`, `loyalt \
  y_tier`, `category_filter`, `total_skus_evaluated`, `floor_violations_count`, `require \
  s_vp_merchandising_override`, and `skus`. Then implement `list_at_risk_vip_orders` que \
  rying `omnichannel_orders` where `status = 'AT_RISK_STOCKOUT'` ordered by `customer_lt \
  v_usd DESC`, returning JSON with `loyalty_tier_filter`, `count`, `total_ltv_exposed_us \
  d`, and `orders`.
```

Phase 4: Full Implementation Prompts

Prompt 4A — Implement `mcp_server/retail_mcp.py`, `.agents/mcp_config.json`, and `.agents/hooks.json`

```
Complete Lab 03 by performing the following changes:
1. In @mcp_server/retail_mcp.py:
   - Implement `simulate_promo_margin_guard(promo_code: str, category: str = "", loyalty \
  _tier: str = "SILVER") -> str` by querying `wms_inventory` for distinct SKUs (filtered \
   by `category` if provided), calling `evaluate_sku_promo(conn, sku, promo_code=promo_c \
  ode, loyalty_tier=loyalty_tier, enforce_floor=True)`, and returning JSON with `promo_c \
  ode`, `loyalty_tier`, `category_filter`, `total_skus_evaluated`, `floor_violations_cou \
  nt`, `requires_vp_merchandising_override`, and `skus`.
   - Implement `list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND", limit: int = 20)  \
  -> str` by querying `omnichannel_orders` where `status = 'AT_RISK_STOCKOUT'` (and `cus \
  tomer_loyalty_tier = ?` unless `loyalty_tier.upper() == "ALL"`), ordered by `customer_ \
  ltv_usd DESC LIMIT ?`, and returning JSON with `loyalty_tier_filter`, `count`, `total_ \
  ltv_exposed_usd`, and `orders`.
2. Create `.agents/mcp_config.json` with a `mcpServers` entry for `retail-pulse-mcp` run \
  ning `.venv/bin/python -m mcp_server.retail_mcp`.
3. Create `.agents/hooks.json` with a `PreToolUse` hook matching `write_file|edit_file|r \
  eplace_file_content` that runs `.venv/bin/python scripts/margin_guard_hook.py`.
```

Phase 5: Verification & Live MCP Queries

Prompt 5A — Run Automated Verification (`verify.sh`)

```
Run `./labs/lab-03-mcp-and-hooks/verify.sh` and confirm all 5 tests in `tests/test_lab03 \
  _mcp_hooks.py` pass, and show the latest entry in `data/logs/hook_compliance_audit.jso \
  nl`.
```

Prompt 5B — Commercial Track: Query the Retail MCP Tools for Executive Insights

```
Using the functions in @mcp_server/retail_mcp.py, answer these three commercial question \
  s:
1. If we stack promo `VIP-EXTRA-15` for `DIAMOND` loyalty members across the `Electronic \
  s` category, how many SKUs hit the 15% margin floor clamp?
2. How many `DIAMOND` VIP orders are currently in `AT_RISK_STOCKOUT` status, and what is \
   the total Customer LTV exposed?
3. Which breaching vendors have an OTIF score below 85%, and what is their combined char \
  geback penalty in USD?
```

Phase 6: Stretch Challenge — Test the Margin Guard Hook Under Adversarial Prompting

Prompt 6 — Attempt to Bypass the 15% Margin Floor (Observe Hook Blocking)

```
Simulate a request from an unauthorized campaign manager: test running `scripts/margin_g \
  uard_hook.py` against a tool payload that tries to edit `src/services/promo_pricing.py \
  ` with `enforce_floor=False`, and another payload that tries to set `"minimum_margin_p \
  ct": 0.05`. Show the hook exit codes, stderr messages, and the audit entries recorded  \
  in `data/logs/hook_compliance_audit.jsonl`.
```